

EXPERIMENTS-CO4-AT2

SUTDENT NAME: P. VISHNU VARDHAN REDDY

REG. NO: 192425212

COURSE CODE: MLA0302

COURSE NAME: REINFORCEMENT LEARNING

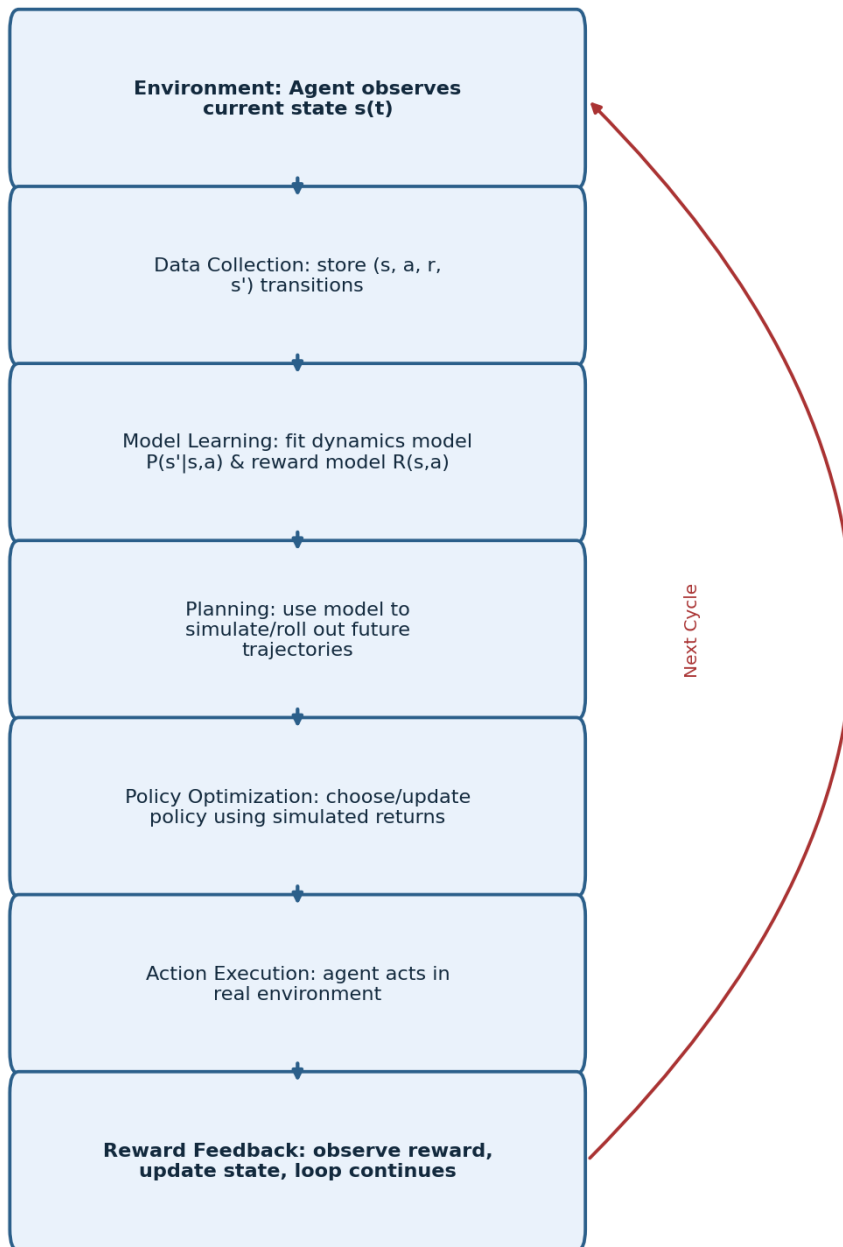
Section 1: Model-Based RL Approach

Q1. Draw a flowchart illustrating the complete workflow of a Model-Based Reinforcement Learning agent from environment interaction to policy optimization.

Answer:

A Model-Based RL agent begins by observing the current state of the environment and collecting transition data (state, action, reward, next state) through interaction. This experience is used to learn an internal model of the environment's dynamics (state-transition function) and reward function. Once a reasonably accurate model is available, the agent uses it to simulate or 'roll out' many possible future trajectories without needing further real-world interaction. These simulated rollouts are evaluated, and the results are used to optimize the policy — i.e., decide which action is best in each state. The optimized policy is then used to select and execute an action in the real environment, the resulting reward and new state are observed, and the cycle repeats, continuously refining both the model and the policy.

Model-Based RL: Complete Workflow

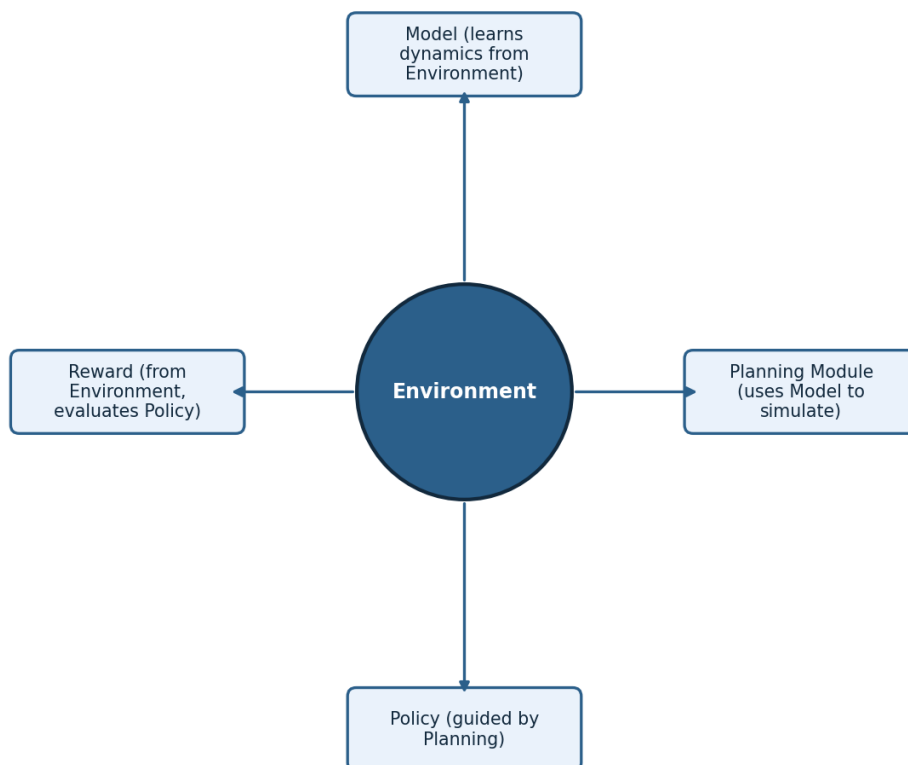


Q2. Draw a concept map showing the relationship between the Environment, Model, Planning Module, Policy, and Reward in Model-Based Reinforcement Learning.

Answer:

The Environment is the central source of experience: it provides states, accepts actions, and returns rewards. The Model is learned directly from Environment interactions and approximates its transition and reward dynamics. The Planning Module consumes the Model to simulate trajectories and compute value estimates without needing real interaction. The Policy is derived from or improved by the Planning Module's output, mapping states to actions. Finally, the Reward signal — generated by the Environment — is used both to train the Model's reward predictor and to evaluate and improve the Policy, closing the loop between all five components.

Concept Map: Core Components of Model-Based RL

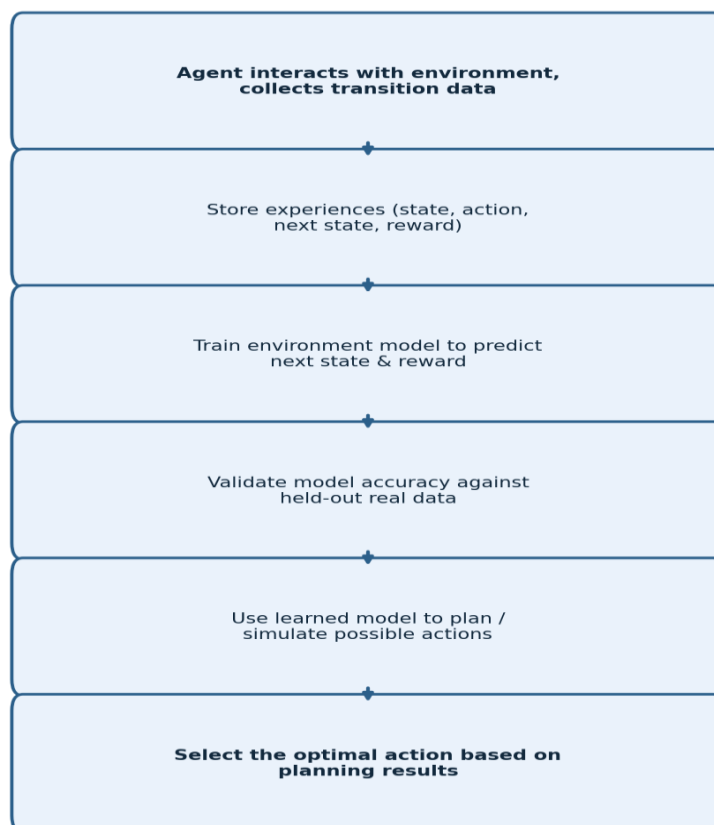


Q3. Design a flowchart explaining how an RL agent learns an environment model before selecting an optimal action.

Answer:

The agent first interacts with the environment to collect experience tuples of state, action, reward, and next state. These are stored and used to train a predictive model — typically a neural network — that approximates the transition function $P(s'|s,a)$ and reward function $R(s,a)$. Before relying on this model, the agent validates its accuracy on held-out data to check that its predictions match real outcomes. Once validated, the model is used for planning: the agent simulates multiple candidate action sequences internally and evaluates their expected outcomes. Based on this planning, the agent selects the optimal action to execute in the real environment.

Learning an Environment Model Before Action Selection

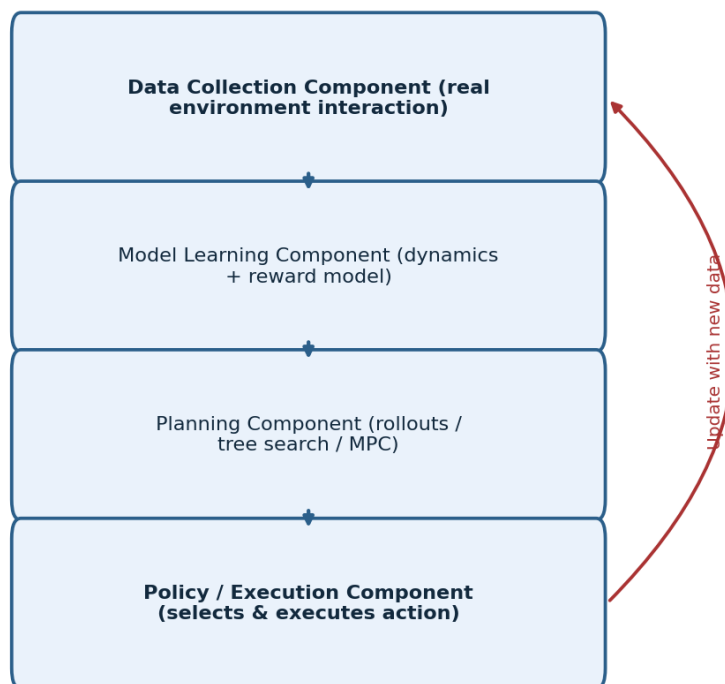


Q4. Draw the architecture of a Model-Based RL system showing model learning, planning, and execution components.

Answer:

The architecture consists of four connected components. The Data Collection component gathers real interaction data from the environment. This feeds the Model Learning component, which fits a dynamics and reward model to the data. The Planning component then uses this model to simulate rollouts, run tree search, or perform model-predictive control (MPC) to evaluate candidate action sequences. Finally, the Policy/Execution component selects and executes the best action in the real environment. New data from execution flows back into Data Collection, forming a continuous feedback loop that keeps improving the model and the resulting policy.

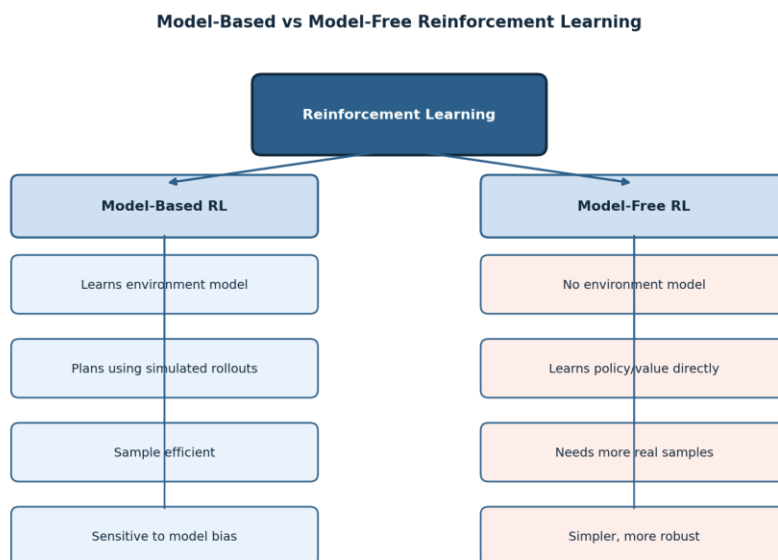
Model-Based RL System Architecture



Q5. Prepare a concept map comparing Model-Based Reinforcement Learning and Model-Free Reinforcement Learning.

Answer:

Both approaches branch from the common goal of Reinforcement Learning. Model-Based RL learns an explicit model of the environment's dynamics, uses it to plan via simulated rollouts, and is generally more sample-efficient — but its performance depends heavily on model accuracy and can suffer from compounding model bias. Model-Free RL, in contrast, learns a policy or value function directly from real experience without building an explicit model; it requires more real-world samples but tends to be simpler to implement and more robust to model errors, since it never relies on a potentially inaccurate simulated environment.



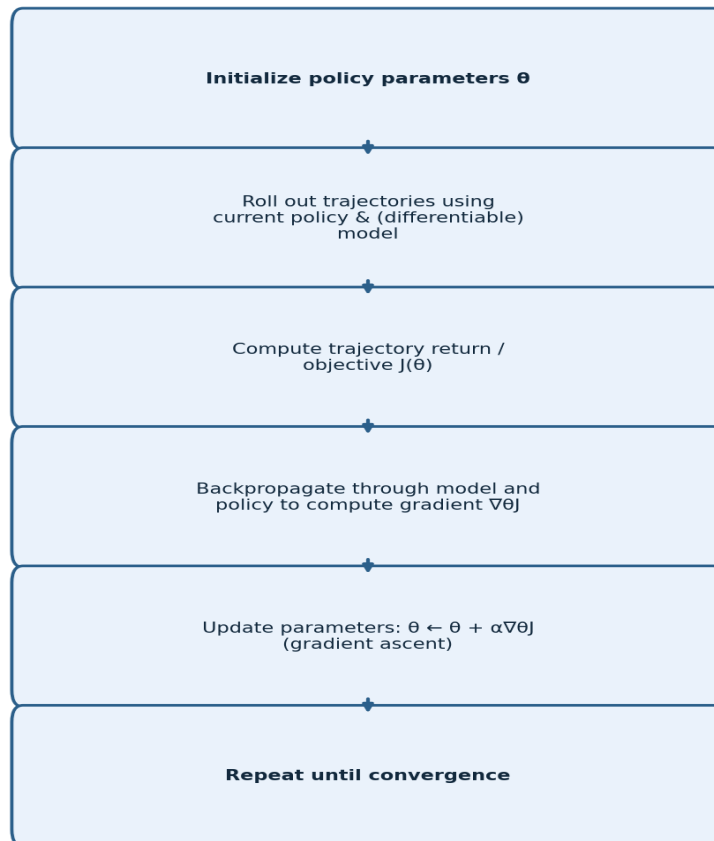
Section 2: Analytic Gradient Computation

Q6. Draw a flowchart explaining the steps involved in analytic gradient computation for policy optimization.

Answer:

Analytic gradient computation begins by initializing the policy parameters θ . The agent then rolls out trajectories using the current policy combined with a differentiable environment model, allowing gradients to flow through both. The trajectory return or objective $J(\theta)$ is computed from these rollouts. Because the model and policy are differentiable, backpropagation can be applied through the entire trajectory to compute the exact analytic gradient $\nabla \theta J$. This gradient is then used to update the policy parameters via gradient ascent, $\theta \leftarrow \theta + \alpha \nabla \theta J$, and the process repeats until the policy converges to a local optimum.

Analytic Gradient Computation for Policy Optimization

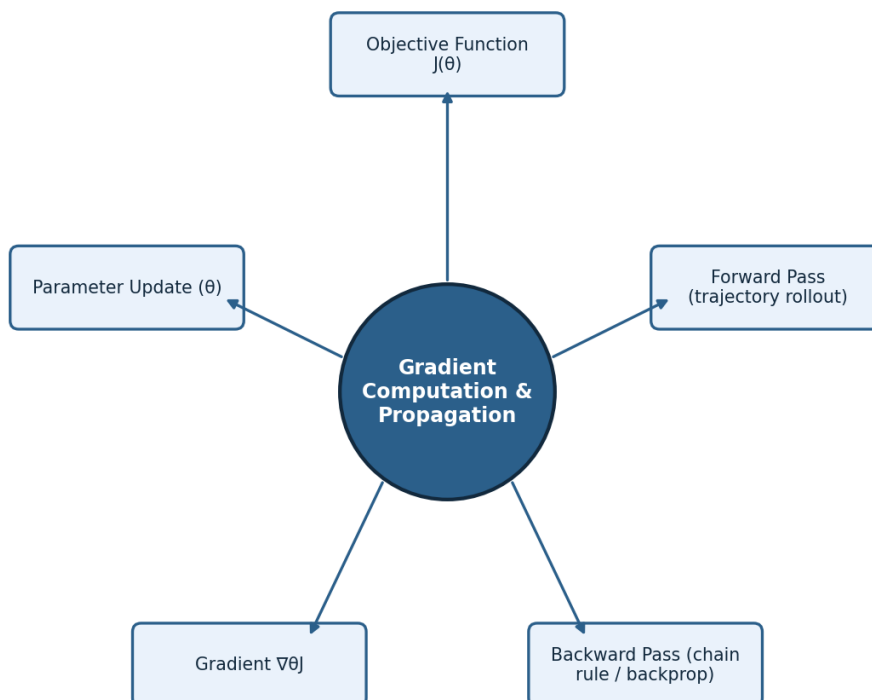


Q7. Design a concept map showing how gradients are computed and propagated during policy learning.

Answer:

The process starts from the Objective Function $J(\theta)$, which measures expected performance. A Forward Pass generates a trajectory rollout using the current policy. A Backward Pass then applies the chain rule (backpropagation) through the trajectory to compute the Gradient $\nabla_{\theta} J$, which quantifies how each parameter affects performance. This gradient is finally used to drive the Parameter Update, adjusting θ in the direction that improves the objective. Each of these stages is tightly linked: the objective determines the rollout, the rollout enables backpropagation, and the resulting gradient dictates the parameter update.

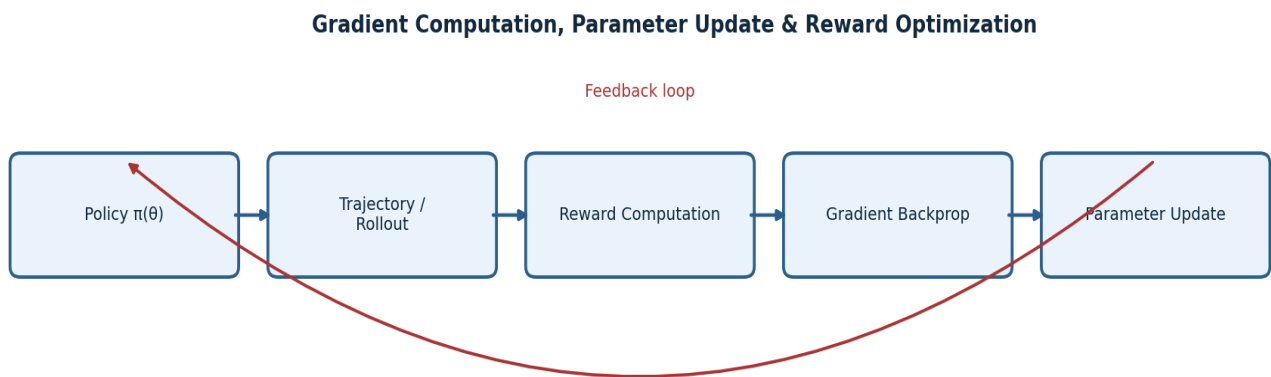
Concept Map: Gradient Computation & Propagation



Q8. Draw a block diagram illustrating gradient computation, parameter updates, and reward optimization in Reinforcement Learning.

Answer:

The block diagram shows a linear pipeline: the current Policy $\pi(\theta)$ generates a Trajectory/Rollout in the environment. The resulting states and actions feed into Reward Computation, which scores the trajectory. This reward signal is used in Gradient Backprop, computing how the policy parameters should change to increase reward. Finally, the Parameter Update block applies this gradient to θ , producing an improved policy for the next iteration. Together these blocks form the core optimization loop that drives reward maximization.

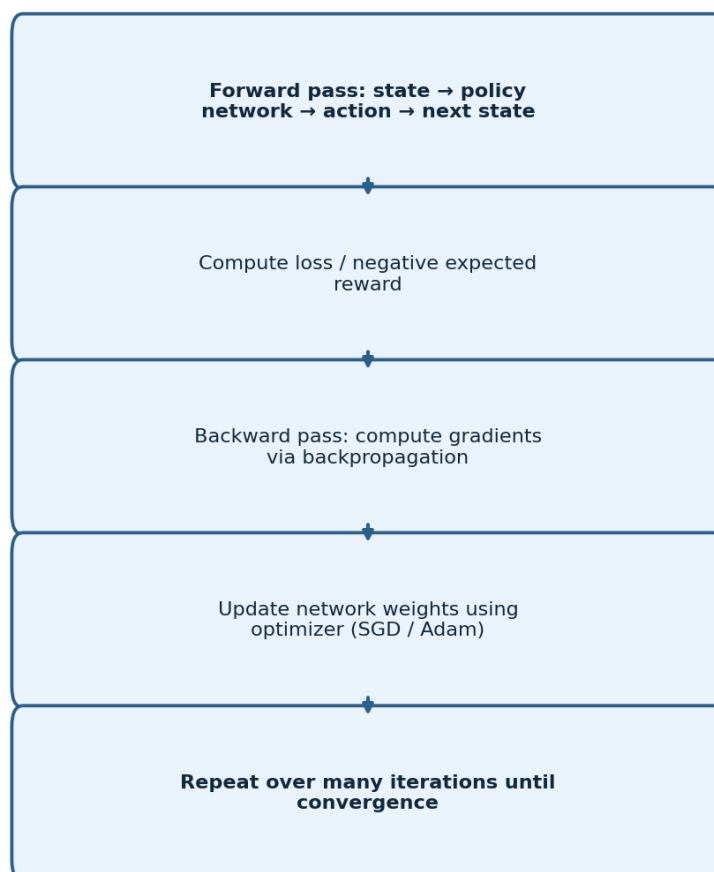


Q9. Prepare a flowchart explaining gradient-based optimization using backpropagation in Reinforcement Learning.

Answer:

The process starts with a forward pass: the current state is passed through the policy network to select an action, which is then applied (via the model or real environment) to produce the next state. A loss function — typically the negative expected reward — is computed from this outcome. A backward pass then applies backpropagation to compute gradients of the loss with respect to every network weight. An optimizer such as SGD or Adam uses these gradients to update the weights, nudging the policy toward higher reward. This forward-backward-update cycle repeats over many iterations until the policy converges.

Gradient-Based Optimization Using Backpropagation

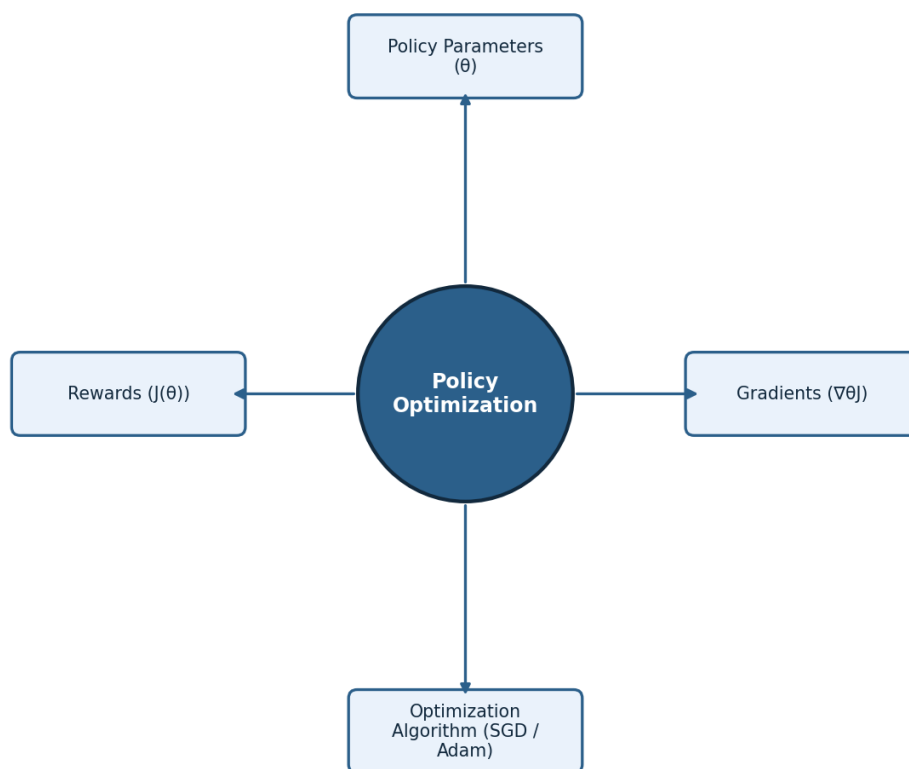


Q10. Draw a concept map showing the relationship among policy parameters, gradients, optimization algorithms, and rewards.

Answer:

Policy Parameters (θ) define the current behavior of the agent and directly determine the Rewards obtained through interaction. Rewards are used to compute Gradients ($\nabla \theta J$), which indicate the direction of parameter change that would increase expected reward. These gradients are consumed by an Optimization Algorithm such as SGD or Adam, which determines the exact update step size and direction. The optimizer then updates the Policy Parameters, closing the loop — improved parameters lead to better rewards, which in turn produce new gradients for the next update.

Policy Parameters, Gradients, Optimization & Rewards



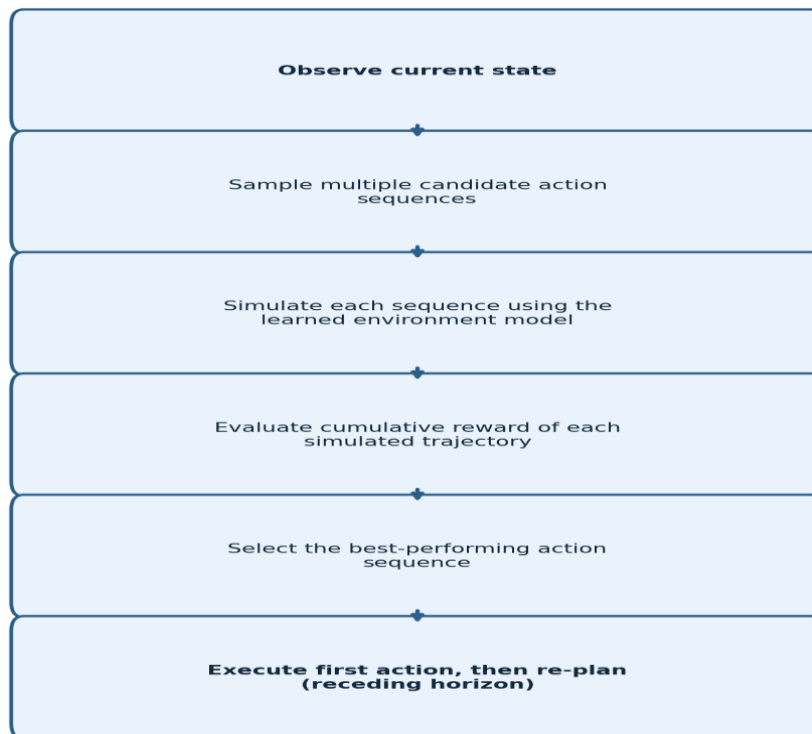
Section 3: Sampling-Based Planning

Q11. Draw a flowchart illustrating the sampling-based planning process in Model-Based Reinforcement Learning.

Answer:

Sampling-based planning begins by observing the current state. The planner samples multiple candidate action sequences (rather than exhaustively enumerating all possibilities). Each sampled sequence is simulated forward using the learned environment model to predict resulting states and rewards. The cumulative reward of each simulated trajectory is evaluated, and the best-performing sequence is selected. Typically only the first action of this best sequence is executed in the real environment (receding-horizon / Model Predictive Control style), after which the agent observes the new state and re-plans, repeating the entire process.

Sampling-Based Planning Process

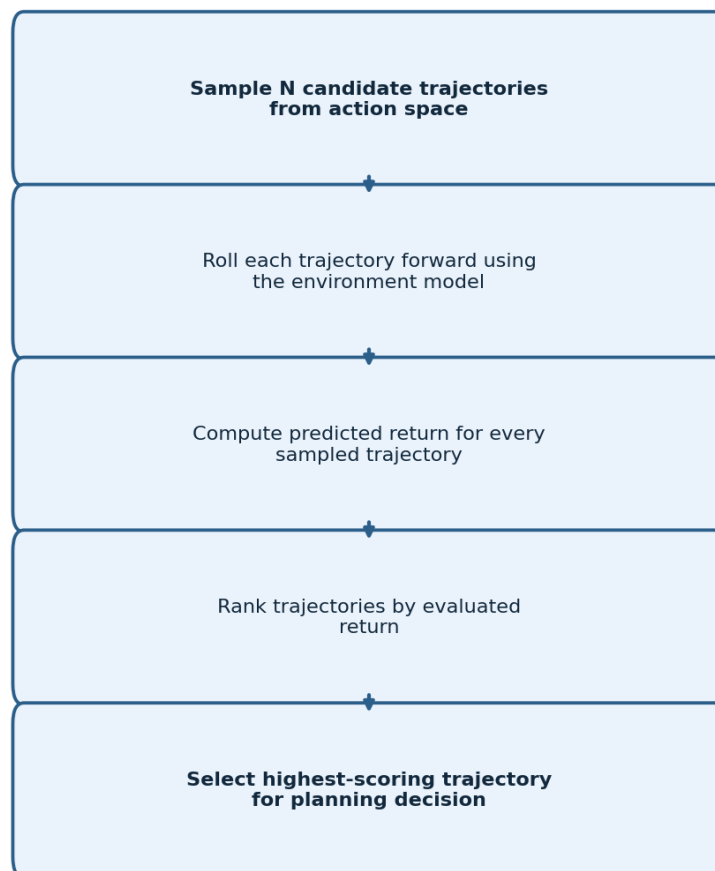


Q12. Prepare a concept map showing how sampled trajectories are generated and evaluated for planning.

Answer:

Planning begins by sampling N candidate trajectories from the space of possible action sequences. Each sampled trajectory is rolled forward using the environment model, producing a simulated sequence of states and rewards. The predicted return of every trajectory is then computed by summing (discounted) rewards along the rollout. These returns are used to rank all sampled trajectories, and the highest-scoring trajectory is selected as the planning decision, guiding the agent's next action.

Sampled Trajectory Generation & Evaluation

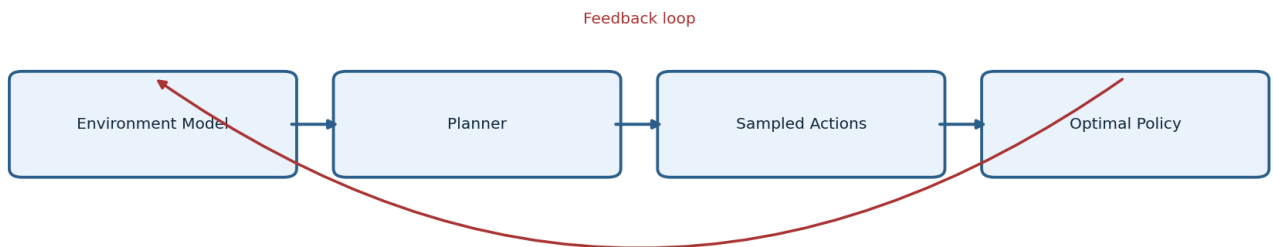


Q13. Draw a block diagram showing the interaction between the environment model, planner, sampled actions, and optimal policy.

Answer:

The Environment Model provides predicted next states and rewards for any given state-action pair. The Planner queries this model repeatedly while considering various Sampled Actions, simulating their outcomes. By comparing the simulated outcomes of different sampled actions, the Planner identifies the sequence that maximizes expected return, which then informs the Optimal Policy — the decision rule the agent ultimately follows when acting in the real environment.

Environment Model, Planner, Sampled Actions & Optimal Policy

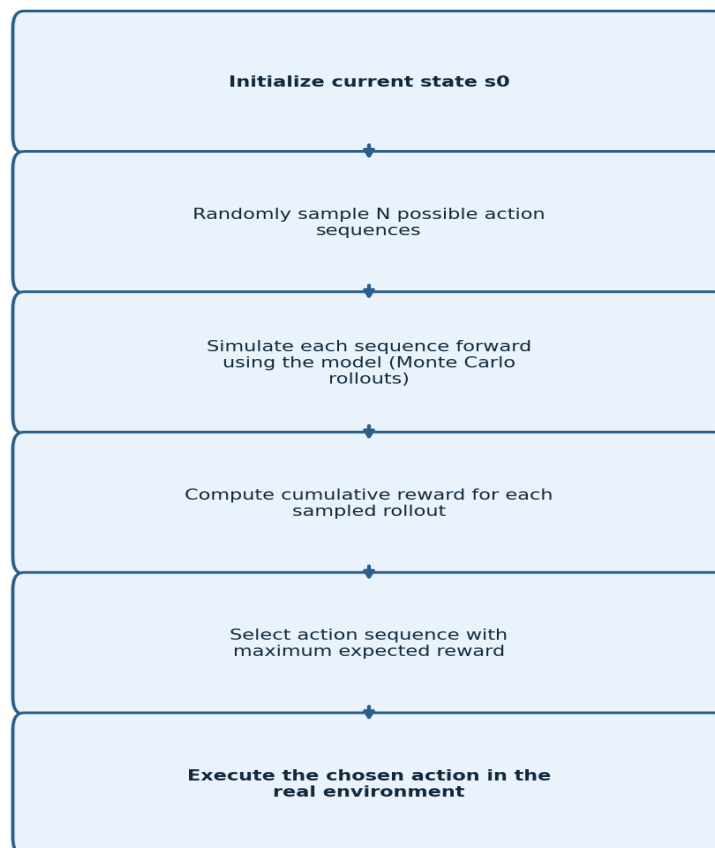


Q14. Design a flowchart explaining Monte Carlo sampling for planning in Reinforcement Learning.

Answer:

Monte Carlo sampling for planning starts by initializing the current state s_0 . The planner randomly samples N possible action sequences from the action space. Each sequence is simulated forward using the environment model — this is the 'Monte Carlo rollout' step, since randomness governs which sequences are tried. The cumulative (discounted) reward is computed for every rollout. The action sequence with the maximum expected reward is selected, and its first action is executed in the real environment, after which the whole sampling process can be repeated for the next decision.

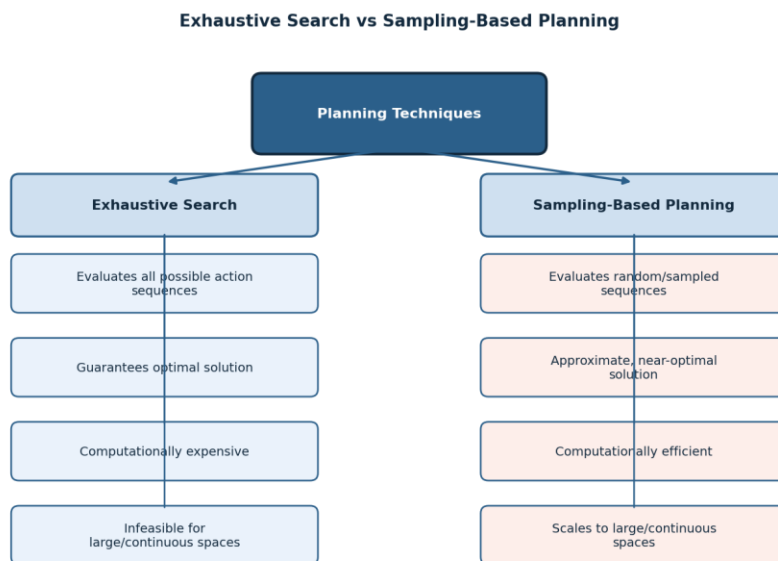
Monte Carlo Sampling for Planning



Q15. Draw a concept map comparing exhaustive search planning and sampling-based planning techniques.

Answer:

Exhaustive Search planning evaluates all possible action sequences systematically, which guarantees finding the optimal solution but is computationally very expensive and becomes infeasible for large or continuous action spaces. Sampling-Based Planning instead evaluates only a random or heuristically chosen subset of sequences, yielding an approximate, near-optimal solution. It is far more computationally efficient and scales well to large or continuous action spaces, making it the practical choice for most real-world Model-Based RL systems, at the cost of an optimality guarantee.



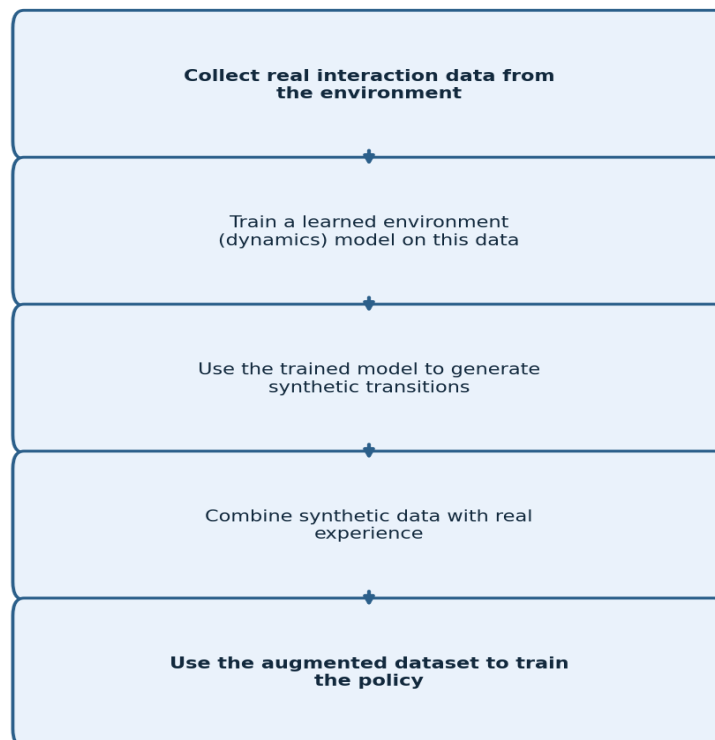
Section 4: Model-Based Data Generation

Q16. Draw a flowchart illustrating synthetic data generation using a learned environment model.

Answer:

The process begins by collecting real interaction data from the environment. This data is used to train a learned environment (dynamics) model that predicts next states and rewards. Once trained, the model is used to generate synthetic transitions by simulating rollouts starting from previously visited states — producing additional (state, action, reward, next state) tuples without any real environment interaction. This synthetic data is combined with the original real experience, and the augmented dataset is used to train the policy, greatly increasing the effective amount of training data available.

Synthetic Data Generation Using a Learned Model

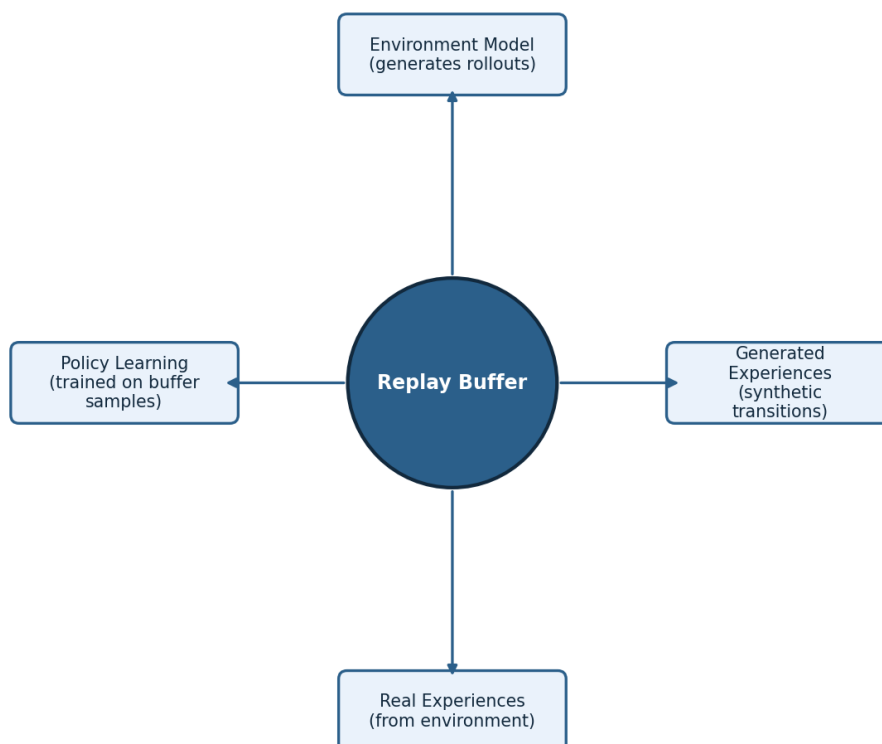


Q17. Prepare a concept map showing the relationship among environment model, generated experiences, replay buffer, and policy learning.

Answer:

The Environment Model, once trained on real data, is used to generate synthetic transitions — the Generated Experiences. These are stored alongside Real Experiences (collected directly from the environment) inside a shared Replay Buffer. Policy Learning then samples mini-batches from this combined Replay Buffer to update the policy or value networks. Because the buffer mixes both real and model-generated data, policy learning benefits from a much larger and more diverse set of training experiences than real interaction alone could provide.

Environment Model, Generated Experiences, Replay Buffer & Policy Learning



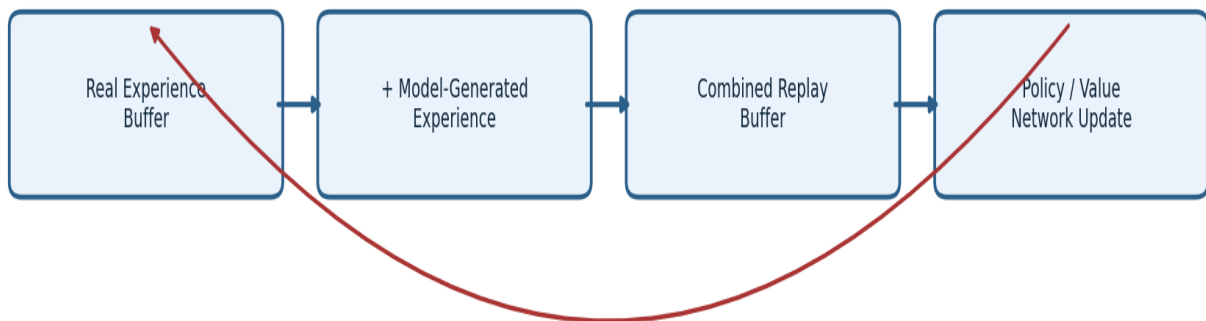
Q18. Draw a block diagram explaining model-generated experience replay in Reinforcement Learning.

Answer:

A Real Experience Buffer stores transitions collected directly from the environment. This is combined with Model-Generated Experience — synthetic rollouts produced by the learned dynamics model — to form a Combined Replay Buffer. This buffer is sampled to perform the Policy/Value Network Update. As the policy improves, it collects better real experience, which in turn improves the model, creating a positive feedback loop between real data collection, model accuracy, and policy quality.

Model-Generated Experience Replay

Improved policy collects more data

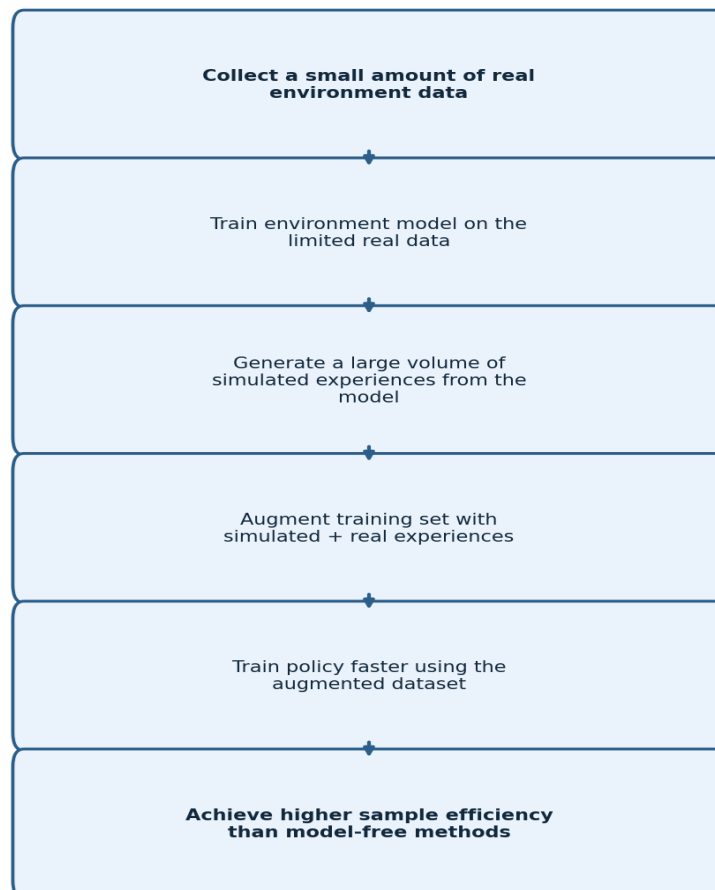


Q19. Design a flowchart showing how simulated experiences improve sample efficiency during training.

Answer:

Training starts by collecting only a small amount of real environment data, since real interaction is often expensive or risky. This limited data is used to train an environment model. The trained model then generates a large volume of simulated experiences at low computational cost. These simulated experiences augment the small real dataset, and the policy is trained on this combined, much larger dataset. Because far more training signal is available per unit of real interaction, the policy converges using fewer real environment samples — i.e., higher sample efficiency — compared to purely model-free methods.

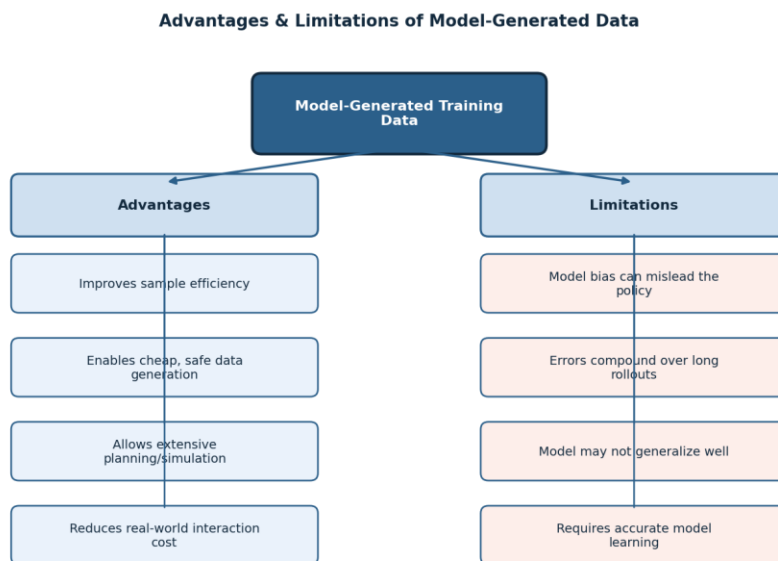
Simulated Experiences Improving Sample Efficiency



Q20. Draw a concept map illustrating the advantages and limitations of model-generated training data.

Answer:

Model-generated training data offers clear Advantages: it improves sample efficiency, allows cheap and safe data generation (no real-world risk), enables extensive planning and simulation, and reduces the cost of real-world interaction. However, it also has important Limitations: model bias can mislead the policy if the learned model is inaccurate, errors compound over long simulated rollouts, the model may not generalize to unseen states, and generating useful synthetic data still requires a sufficiently accurate model to begin with — creating a trade-off that practitioners must manage carefully.



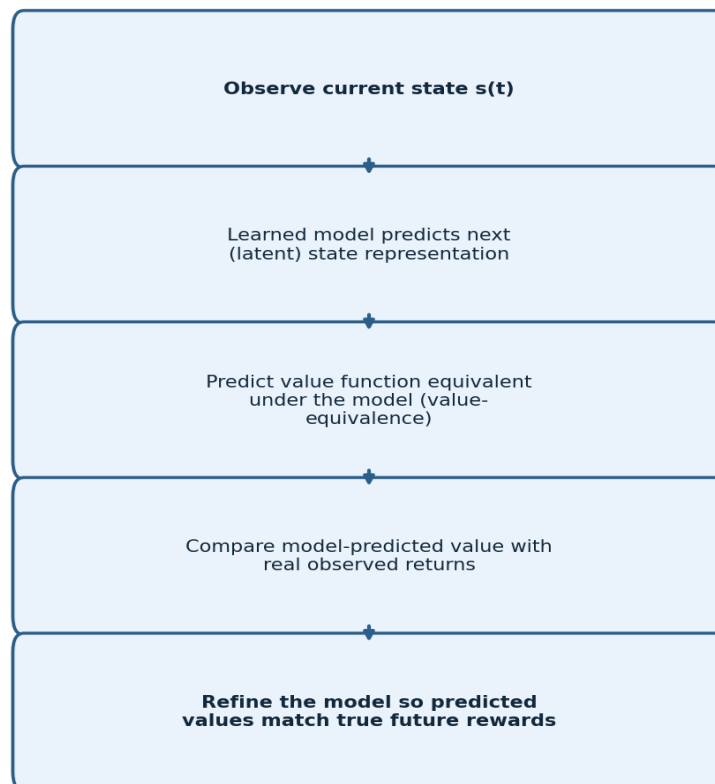
Section 5: Value-Equivalence Prediction and Policy Optimization

Q21. Draw a flowchart explaining value-equivalence prediction for estimating future rewards.

Answer:

Value-equivalence prediction starts by observing the current state $s(t)$. The learned model predicts a next (often latent) state representation rather than a full pixel-perfect reconstruction. From this predicted representation, the model estimates a value function that is 'value-equivalent' to the true environment — meaning it need only preserve information relevant to predicting future rewards and values, not every detail of the state. This predicted value is compared against real observed returns, and the discrepancy is used to refine the model so its value predictions increasingly match true future rewards, even though the model itself may not reconstruct the environment exactly.

Value-Equivalence Prediction for Estimating Future Rewards

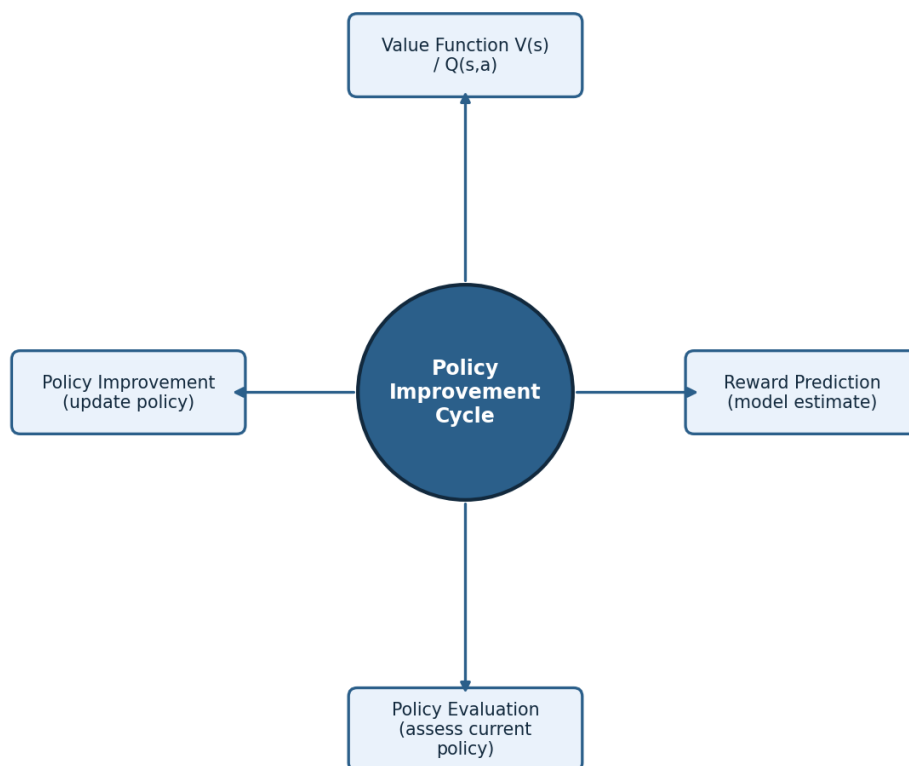


Q22. Prepare a concept map showing the relationship among value functions, reward prediction, policy evaluation, and policy improvement.

Answer:

The Value Function $V(s)$ or $Q(s,a)$ estimates expected long-term return and depends on accurate Reward Prediction from the model. Policy Evaluation uses the value function to assess how good the current policy is in each state. Based on this assessment, Policy Improvement updates the policy to favor actions with higher estimated value. The improved policy then generates new experience, which refines reward prediction and the value function again — forming the classic generalized policy iteration cycle at the heart of value-based reinforcement learning.

Value Functions, Reward Prediction, Policy Evaluation & Improvement

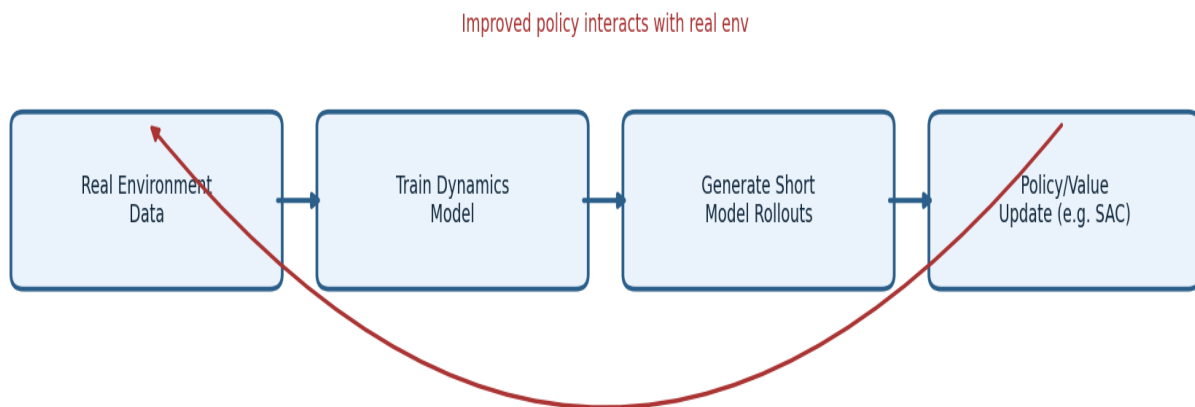


Q23. Draw a block diagram illustrating the complete Model-Based Policy Optimization (MBPO) framework.

Answer:

MBPO begins with Real Environment Data collected by the current policy. This data is used to Train the Dynamics Model. The trained model then generates short, branched Model Rollouts starting from real states — kept short to limit compounding model error. These short synthetic rollouts, combined with real data, are used for a Policy/Value Update using a model-free algorithm such as Soft Actor-Critic (SAC). The improved policy then interacts with the real environment again, collecting new real data and closing the loop, which is the essence of MBPO's efficient hybrid model-based/model-free design.

Model-Based Policy Optimization (MBPO) Framework

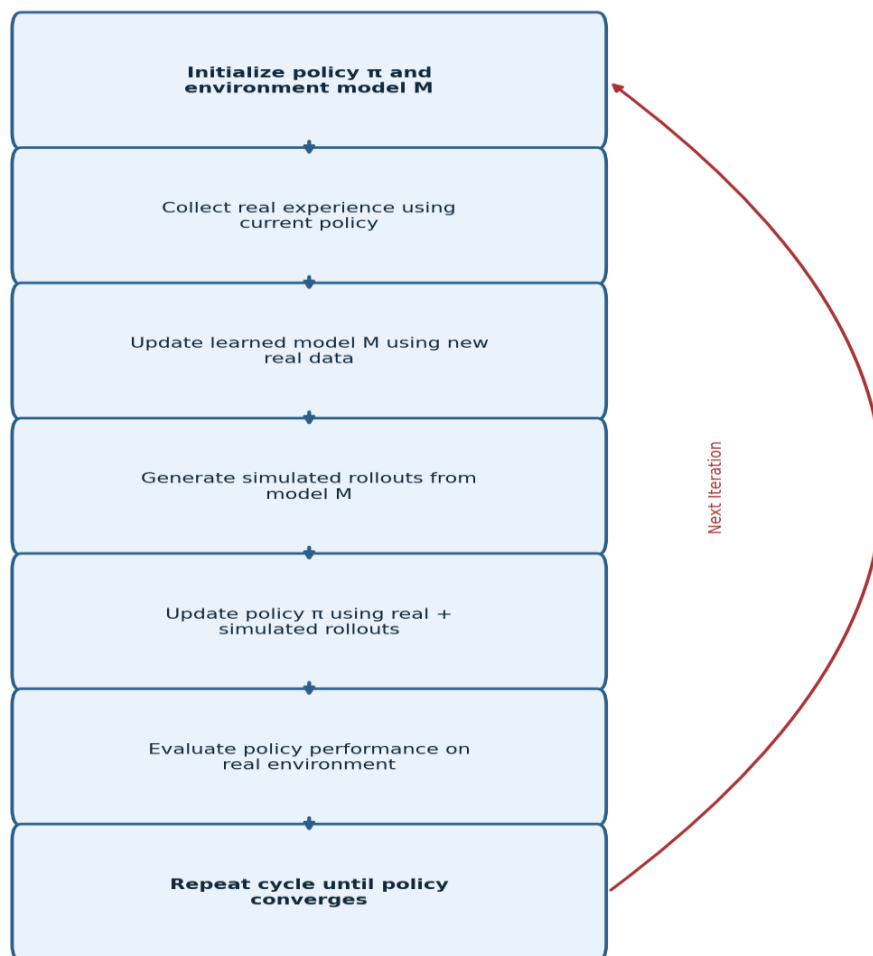


Q24. Design a flowchart explaining iterative policy optimization using learned environment dynamics.

Answer:

The process begins by initializing both a policy π and an environment model M . The agent collects real experience using the current policy, and this data is used to update the model M . The updated model is then used to generate simulated rollouts. The policy π is updated using a combination of real and simulated rollouts, improving faster than would be possible with real data alone. The updated policy is evaluated on the real environment to check for actual performance gains, and the entire cycle — collect, update model, simulate, update policy, evaluate — repeats until the policy converges to satisfactory performance.

Iterative Policy Optimization Using Learned Dynamics



Q25. Draw a comprehensive concept map integrating Model-Based RL, Analytic Gradient Computation, Sampling-Based Planning, Model-Based Data Generation, Value-Equivalence Prediction, and Model-Based Policy Optimization into a unified Reinforcement Learning framework.

Answer:

At the center of the unified framework lies Model-Based RL as the core approach: learning and using an environment model. Analytic Gradient Computation branches off as one optimization strategy, using differentiable models to compute exact gradients for policy updates. Sampling-Based Planning is an alternative (or complementary) decision-making strategy, using the model to evaluate sampled action sequences instead of exact gradients. Model-Based Data Generation shows how the same learned model can be reused to produce synthetic training data, improving sample efficiency for any downstream learning algorithm. Value-Equivalence Prediction refines what the model should predict — favoring representations that preserve value-relevant information over exact state reconstruction. Finally, Model-Based Policy Optimization (MBPO) integrates all of these ideas into a practical algorithm: it learns a model, generates short synthetic rollouts, and combines them with real data to optimize the policy efficiently — demonstrating how each concept studied individually contributes to a single, cohesive Model-Based Reinforcement Learning pipeline.

